IMU Calibration and Attitude Estimation for a Four-Wheel Skid-Steer Robot

Guided Test Firmware (`IMU_CALIBRATE.cpp`), Attitude Filter (`attitude.h`), Sensor Driver, and Integration into the Odometry Chain

[Author name]
[Institution / Department]

2026-10-17

Abstract

The heading that the robot publishes in `/odometry/filtered` is the time integral of one gyroscope axis, so every error of that gyroscope appears directly in the pose that Nav2 and the SLAM stack receive. This report documents how the inertial sensor, an MPU6050 on I^2C, is characterised, corrected and used. It covers four parts: the guided test firmware `IMU_CALIBRATE.cpp`, which in four stages (still, one full turn, nose up, left side up) measures offsets, noise, gyro scale and sign conventions; the sensor driver `MPU6050_Custom`, which applies stored static offsets and a boot-time gyro bias; the roll and pitch complementary filter `attitude.h`, with its accelerometer gain correction, gravity compensation and ramp detector; and the way `main.cpp` combines these with a zero-velocity update, a dead-band and a four-state extended Kalman filter. The report gives the error models and the equations behind each step, the exact operating procedure with acceptance criteria and formulas for updating the stored constants, a validation record from hand-turn tests, Python replicas of the filter and of the EKF heading path that check specific claims, a troubleshooting guide, and a list of limitations found while reading the code. Result tables and photo slots are left blank for the commissioning data.

Contents

1 Introduction **7**

1.1 Motivation . . . 7

1.2 Scope . . . 7

1.3 Conventions and notation . . . 7

1.4 Precautions . . . 8

2 System Description **9**

2.1 Hardware . . . 9

2.2 Axes and sign conventions . . . 10

2.3 Sensor configuration . . . 10

2.4 The driver `MPU6050_Custom` . . . 11

2.4.1 Static offsets and scaling . . . 11

2.5 Boot sequence . . . 12

3 Theory **13**

3.1 Error model of a MEMS gyroscope . . . 13

3.2 Bias estimation, ZUPT and dead-band . . . 13

3.2.1 Boot-time estimate . . . 13

3.2.2 Zero-velocity update . . . 14

3.2.3 Dead-band . . . 14

3.3 Accelerometer tilt sensing . . . 14

3.4 Euler-angle kinematics . . . 15

3.5 The complementary filter . . . 15

3.5.1 Effect of a gyro bias . . . 15

3.5.2 Accelerometer gating . . . 16

3.6 Gravity compensation and its side effect . . . 16

3.7 Heading integration and the EKF . . . 16

3.8 Error budget . . . 17

4 Test Firmware **18**

4.1 Purpose and design . . . 18
4.2 Building and flashing . . . 18
4.3 Constants and data structures . . . 19
4.4 Sampling and statistics . . . 19
4.5 Start-up probes . . . 19
4.6 Stage 1: still . . . 20
4.7 Stage 2: spin . . . 20
4.8 Stages 3 and 4: tilt . . . 21
4.9 Output reference . . . 22

5 Attitude Filter **23**

5.1 Interface . . . 23
5.2 Calibration . . . 23
5.3 The update step . . . 24
5.4 Ramp detection . . . 24
5.5 Behaviour checked by simulation . . . 25
5.6 Limitations . . . 25

6 Integration in main.cpp **26**

6.1 Boot . . . 26
6.2 Processing order in the 20 Hz loop . . . 26
6.3 ZUPT and dead-band code . . . 27
6.4 The heading path in the EKF . . . 27
6.5 Published messages . . . 28
6.6 The self-check . . . 28
6.7 Risk at power-up . . . 28

7 Procedure **29**

7.1 Before you start . . . 29
7.2 Running the guided test . . . 29
7.3 Acceptance criteria . . . 30
7.4 Updating the stored constants . . . 30
7.5 Multi-turn scale test . . . 31
7.6 Verifying after a change . . . 31

8 Validation **33**

8.1 Hand-turn tests on the robot . . . 33

8.2 Phone compass comparison 33
8.3 Replica of the attitude filter 34
8.4 Data to collect on the robot 34

9 Troubleshooting **36**

10 Limitations **37**
10.1 Properties of the method 37
10.2 Findings in the code 37
10.3 Possible improvements 37

A Quick Reference **39**

B Simulation Scripts **40**

List of Figures

1.1 Robot on a level table with the motor battery disconnected, ready for the IMU test. 8

2.1 The IMU board on the chassis, with the three body axes drawn on the photo. . . 9

2.2 Body frame (FLU) seen from above. Rotation about z is positive counter-clockwise. 10

2.3 Power-up sequence of the IMU in `setup()`. About four seconds of stillness are needed. 12

3.1 Magnitude of the two paths of the complementary filter for $\tau = 1\,\mathrm{s}$. They cross at $0.159\,\mathrm{Hz}$ (dashed), where each contributes $-3\,\mathrm{dB}$. 15

3.2 Simulation: velocity from integrating the compensated forward acceleration (dashed) against the true velocity, for $0.3\,\mathrm{m/s^2}$ applied between 1 and 5 s. 16

7.1 Test set-up: robot on the marked surface, laptop with serial monitor, block for the tilt stages 29

7.2 Serial monitor output of a complete run, showing the RESULT lines 30

7.3 Floor line and robot nose mark for the multi-turn test 31

8.1 Stage 2 trace: integrated gyro angle against time for one counter-clockwise turn . 34

8.2 Pitch estimate against time while lifting the nose by 15° (Stage 3), with the accelerometer angle for comparison 34

8.3 Dashboard yaw against time during a 5-minute still test after boot 35

List of Tables

1.1 Symbols used in this report. 8

2.1 Hardware summary. Items marked [TODO:] are for the builder to complete. . 9

2.2 Expected signs. Right-hand rule about each axis: rotation about $+x$ lifts the left side, about $+y$ pitches the nose down, about $+z$ turns counter-clockwise. 10

2.3 MPU6050 configuration used by the driver. Values for the default registers and the filter bandwidths are from my recollection of the datasheet; please check them against it. 10

2.4 Functions of `MPU6050_Custom`. 11

2.5 Static constants in `readData`. The measured value is the quantity subtracted, so for example a_x gets $+0.377\,\mathrm{m/s^2}$. 12

3.1 Gyro error terms and their size for this robot. Datasheet figures are quoted as I remember them and must be checked; the measured values come from Chapter 8. 13

3.2 Heading error budget. ψ is the angle turned and T the elapsed time. 17

4.1 Constants and state in `IMU_CALIBRATE.cpp`. 19

4.2 Start-up probe lines and their interpretation. Register behaviour is from my recollection of the datasheets and register maps; check them. 19

4.3 The `RESULT` lines, in the order they appear. 22

5.1 Public interface of the `Attitude` class. 23

5.2 Constants in `attitude.h`. 24

5.3 Steady-state pitch error caused by a residual gyro bias (level, still robot, $\tau = 1\,\mathrm{s}$). The simulated value equals the prediction τb. 25

6.1 Order of operations on each IMU sample in `main.cpp`. 27

6.2 Largest difference between the EKF heading and $\int g_z\,dt$ over 60 s (simulation). . 28

7.1 Acceptance criteria for the guided test. Thresholds are practical limits chosen for this robot, not datasheet values. 30

7.2 Commissioning record. Fill in at each calibration. 32

8.1 Five-turn tests. Single runs, floor line aligned by eye (± 1 to 2°). 33

8.2 Simulation checks of the attitude filter and EKF heading path. 34

9.1 Symptoms, likely causes and checks. 36

A.1 Quick reference. 39

Chapter 1

Introduction

1.1 Motivation

A wheeled robot that maps and navigates autonomously needs a pose estimate whose errors stay small between the moments when the lidar corrects it. On a skid-steer chassis the wheels slip sideways during every turn, so wheel odometry under-reads rotation by an amount that depends on the floor. The gyroscope does not have that problem, which is why the firmware takes yaw from the gyro alone. The price is that the gyro's own errors, a scale error, a residual bias and noise, are no longer averaged against anything else. A scale error of 0.4 % is 1.5° per full turn; a residual bias of 0.001 rad/s is 3.4° per minute of driving. Both are small, and both are worth measuring rather than assuming.

The same sensor provides roll and pitch, used to remove gravity from the forward acceleration that feeds the velocity estimate and to flag when the robot is on a ramp.

1.2 Scope

This document covers:

- the guided test firmware `IMU_CALIBRATE.cpp` (its header comment calls it `imu_guided_test.cpp`, see Chapter 10);
- the attitude filter `attitude.h`;
- the driver `MPU6050_Custom.h/.cpp`;
- the IMU, ZUPT, dead-band and EKF heading path in `main.cpp`.

It does not cover the lidar, SLAM, or the wheel speed loops (see the companion report on `PID_TEST.cpp` and `motor_calibrate.py`). The robot has no magnetometer, so absolute heading is not available and heading is always relative to the pose at power-up.

1.3 Conventions and notation

Frames and signs follow ROS REP-103: the body frame is *forward–left–up* (FLU), right-handed, with x forward, y to the left and z up. Angles are in radians internally and degrees in console output. Table 1.1 lists the symbols.

Table 1.1: Symbols used in this report.

Symbol	Unit	Meaning
a_x, a_y, a_z	m/s^2	Specific force in the body frame (accelerometer output)
p, q, r (or g_x, g_y, g_z)	rad/s	Body angular rates about x, y, z
ϕ	rad	Roll, positive when the left side is up
θ	rad	Pitch, positive when the nose is down
ψ	rad	Yaw, positive counter-clockwise seen from above
G	m/s^2	Standard gravity, 9.80665
k (`k_`)	–	Accelerometer gain correction, $G/\|\mathbf{a}\|$ at calibration
b, $\hat{b}$	rad/s	True and estimated gyro bias
s	–	Gyro scale error: measured rate $= (1 + s)\times$ true rate
τ	s	Complementary-filter time constant, 1.0
α	–	Gyro weight per step, $\tau/(\tau + \Delta t)$
Δt	s	Sample period: 0.05 in `main.cpp`, 0.02 in the test
λ	–	ZUPT gain per sample, 0.02
N_{ARW}	rad/$\sqrt{\text{s}}$	Angle random walk coefficient

1.4 Precautions

Warning. `IMU_CALIBRATE.cpp` contains no motor code and never configures the motor driver pins, so nothing is driven. Even so, an unconfigured driver input can float, and some drivers respond to that. Disconnect the motor battery, or put the robot on a stand with the wheels free, before flashing. The spin stage is done by hand: keep the robot flat on the surface, and keep the USB cable slack so that it does not drag on the robot or pull the board.

PHOTO PLACEHOLDER

Robot on a level table with the motor battery disconnected, ready for the IMU test.

Figure 1.1: Robot on a level table with the motor battery disconnected, ready for the IMU test.

Chapter 2

System Description

2.1 Hardware

The inertial sensor is an MPU6050 (3-axis gyroscope and 3-axis accelerometer) read over I^2C at address 0x68 through the `Wire` library on the pins `I2C_SDA` and `I2C_SCL` from `pins.h`. It shares the bus with the TCS34725 colour sensor. Table 2.1 lists the facts the report relies on; the rest is to be filled in.

Table 2.1: Hardware summary. Items marked **[TODO:]** are for the builder to complete.

Item	Value
Microcontroller	ESP32-S3 (Arduino framework, PlatformIO)
IMU	MPU6050, I^2C 0x68; breakout **[TODO: board name, genuine or clone]**
Mounting	**[TODO: location on the chassis, orientation, how fixed, vibration isolation]**
Axes	Sensor frame equals the robot FLU frame (no axis flips in the firmware)
Bus clock	Arduino default of `Wire` (I think 100 kHz; the code does not set it)
Gyro range	$\pm 250^\circ$/s (± 4.36 rad/s); the robot's top rate is about 2.25 rad/s
Accelerometer range	$\pm 2\,g$

PHOTO PLACEHOLDER

The IMU board on the chassis, with the three body axes drawn on the photo.

Figure 2.1: The IMU board on the chassis, with the three body axes drawn on the photo.

2.2 Axes and sign conventions

Figure 2.2 shows the body frame. Every sign in the code, in the test prompts and in this report follows from the right-hand rule applied to it, so it is worth stating once. Table 2.2 gives the expected sign of each sensor channel for each motion.

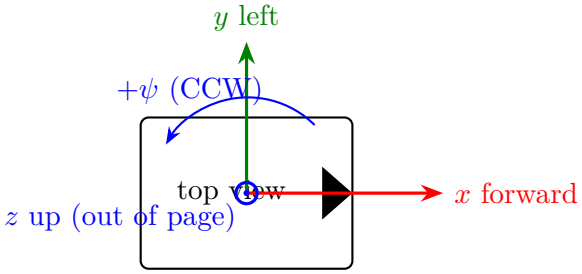


Figure 2.2: Body frame (FLU) seen from above. Rotation about z is positive counter-clockwise.

Table 2.2: Expected signs. Right-hand rule about each axis: rotation about $+x$ lifts the left side, about $+y$ pitches the nose down, about $+z$ turns counter-clockwise.

Motion	a_x	a_y	a_z	Gyro axis	Gyro sign
Nose up (front lifted)	+	0	$\approx G\cos$ (slightly less)	g_y	−
Left side up	0	+	slightly less	g_x	+
Turn counter-clockwise (flat)	0	0	G	g_z	+
Speeding up forwards (level)	+	0	G	–	–

For a robot at rest with roll ϕ and pitch θ (nose down positive), the specific force in the body frame is

$$\begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} = G \begin{pmatrix} -\sin\theta \\ \cos\theta\sin\phi \\ \cos\theta\cos\phi \end{pmatrix}, \tag{2.1}$$

so nose up (negative θ) gives a positive a_x, and left side up gives a positive a_y, as in the table. Note that a stationary accelerometer reads $+G$ on the axis pointing up, not $-G$.

2.3 Sensor configuration

The driver writes only two registers; everything else keeps its power-on default. Table 2.3 lists the configuration that results.

Table 2.3: MPU6050 configuration used by the driver. Values for the default registers and the filter bandwidths are from my recollection of the datasheet; please check them against it.

Register	Value	Effect
`PWR_MGMT_1` (0x6B)	0x00	Wakes the chip, internal 8 MHz oscillator as clock
`CONFIG` (0x1A)	0x03	Digital low-pass filter: about 44 Hz accelerometer, 42 Hz gyro, gyro output rate 1 kHz
`GYRO_CONFIG` (0x1B)	0x00 (default)	±250°/s, 131 LSB per °/s
`ACCEL_CONFIG` (0x1C)	0x00 (default)	±2 g, 16384 LSB per g
`SMPLRT_DIV` (0x19)	not written	Internal sample rate not divided; the firmware simply reads the latest value

One LSB of the gyro is $1/131 = 0.00763^\circ/\mathrm{s} = 1.33\times10^{-4}\,\mathrm{rad/s}$, and one LSB of the accelerometer is $6.0\times10^{-4}\,\mathrm{m/s^2}$. Both are far below the noise, so quantisation of the raw data does not matter for heading.

Sampling and aliasing. `main.cpp` reads the sensor every 50 ms (20 Hz) and the test firmware every 20 ms (50 Hz). Both rates have a Nyquist frequency (10 Hz and 25 Hz) below the 42 Hz corner of the sensor's own filter, so vibration between those frequencies is aliased into the measured band instead of being removed. This adds noise but, being roughly zero-mean, no bias to the integral; Chapter 10 comes back to it.

2.4 The driver `MPU6050_Custom`

Table 2.4 lists the driver's functions. The raw read is 14 bytes from register 0x3B in the order a_x, a_y, a_z, temperature, g_x, g_y, g_z, each a big-endian 16-bit integer; the driver stores them in `v[0..6]` so that the gyro is `v[4..6]` and `v[3]` (temperature) is read but never used.

Table 2.4: Functions of `MPU6050_Custom`.

Function	Behaviour
`begin()`	Writes `PWR_MGMT_1`=0 and `CONFIG`=0x03; returns false only if the first write is not acknowledged
`readRaw(v)`	14-byte burst read; false on any bus error
`calibrateGyro(400)`	Averages raw g_z over 400 reads (3 ms delay each) and stores it as `gyro_bias_z` in rad/s; only accepted if at least a quarter of the reads succeeded
`readData(...)`	Scales to SI units, subtracts the static offsets of Table 2.5 and the runtime g_z bias
`healthCheck()`	WHO_AM_I must be 0x68, 0x70, 0x71 or 0x72; the mean of 20 raw $\lvert\mathbf{a}\rvert$ readings must lie between 13100 and 19660 counts (0.8 to 1.2 g); and $\lvert\hat{b}_z\rvert \le 0.35\,\mathrm{rad/s}$
`getGyroBiasZ()`, `updateGyroBiasZ()`	Access to the runtime g_z bias, used by the ZUPT in `main.cpp`

2.4.1 Static offsets and scaling

With v_i the raw count, `readData` computes

$$a_x = \tfrac{v_0}{16384}G - c_{ax}, \qquad a_y = \tfrac{v_1}{16384}G - c_{ay}, \qquad a_z = \tfrac{v_2}{16384}G - c_{az}, \tag{2.2}$$

$$g_x = \tfrac{v_4}{131}\tfrac{\pi}{180} - c_{gx}, \qquad g_y = \tfrac{v_5}{131}\tfrac{\pi}{180} - c_{gy}, \qquad g_z = \tfrac{v_6}{131}\tfrac{\pi}{180} - \hat{b}_z, \tag{2.3}$$

with the constants of Table 2.5. These are the "stage 1 resting offsets" that the code comments refer to: values measured once, on one board, at one temperature, with the robot at rest. There is no scale correction for g_z in the code, although a comment in `readData` still mentions a "Stage 2 Z-axis scale factor (0.793)"; that comment is stale (Chapter 10).

Table 2.5: Static constants in `readData`. The measured value is the quantity subtracted, so for example a_x gets $+0.377\,\mathrm{m/s^2}$.

Channel	Constant	Value	Unit
a_x	c_{ax}	-0.377	$\mathrm{m/s^2}$
a_y	c_{ay}	0.211	$\mathrm{m/s^2}$
a_z	c_{az}	$10.444 - G = 0.637$	$\mathrm{m/s^2}$
g_x	c_{gx}	-0.04539 (equal to $-2.60°/\mathrm{s}$)	rad/s
g_y	c_{gy}	0.01483 ($0.85°/\mathrm{s}$)	rad/s
g_z	–	measured at every boot by `calibrateGyro`	rad/s

The a_z constant tells us that this board's raw z reading at rest is $10.444\,\mathrm{m/s^2}$, which is 6.5 % above G. The driver removes that excess as an offset, and `attitude.h` later removes any remaining gain error with a single factor (Chapter 5). One static pose cannot separate an offset from a gain error on a_z; Section 3.3 explains the consequence.

2.5 Boot sequence

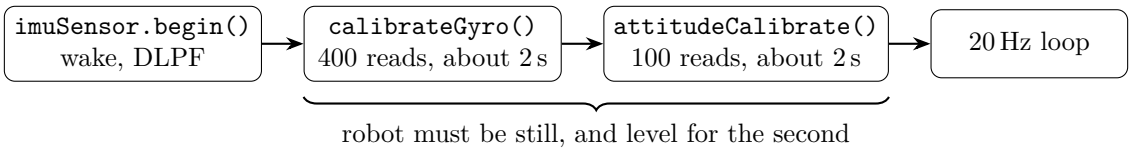


Figure 2.3: Power-up sequence of the IMU in `setup()`. About four seconds of stillness are needed.

The gyro calibration takes about 1.2 s of delays plus the I^2C time of 400 reads, and the attitude calibration about 2 s (100 reads with a 20 ms delay). I estimate four seconds in total; I did not time it. If the robot is moved during either step, the bias or the zero attitude is wrong from then on (Section 6.7).

Chapter 3

Theory

3.1 Error model of a MEMS gyroscope

A gyroscope channel measures

$$\omega_{\text{meas}}(t) = (1 + s)\,\omega(t) + b(t) + n(t), \tag{3.1}$$

where s is the scale error, $b(t)$ a slowly varying bias and $n(t)$ white noise. Further terms exist (cross-axis misalignment, sensitivity to acceleration and vibration, temperature dependence of both s and b), but for a robot that turns about a single axis on a flat floor the three terms above dominate. Integrating the corrected measurement over a time T gives the heading error

$$\hat{\psi}(T) - \psi(T) = s\,\psi(T) + \int_0^T (b - \hat{b})\,dt + N_{\text{ARW}}\,\sqrt{T}\,\xi, \qquad \xi \sim \mathcal{N}(0,1). \tag{3.2}$$

The first term grows with the angle turned, the second with time, and the third with the square root of time. Table 3.1 gives the orders of magnitude for this robot.

Table 3.1: Gyro error terms and their size for this robot. Datasheet figures are quoted as I remember them and must be checked; the measured values come from Chapter 8.

Term	Datasheet (check)	This robot
Scale error s	about $\pm 3\,\%$ tolerance	$+0.43\,\%$ measured, i.e. 1.5° per turn
Zero-rate offset	about $\pm 20^\circ/\text{s}$ initially	removed at boot; the driver rejects $\lvert b\rvert > 0.35\,\text{rad/s}$, which is $20^\circ/\text{s}$
Residual bias $b - \hat{b}$	–	$10^{-3}\,\text{rad/s}$ gives 3.4° per minute
Noise density	about $0.005^\circ/\text{s}/\sqrt{\text{Hz}}$	0.04° after one minute: negligible

3.2 Bias estimation, ZUPT and dead-band

3.2.1 Boot-time estimate

`calibrateGyro` averages $n = 400$ raw readings of g_z. If the samples were independent with standard deviation σ, the error of the mean would be $\sigma/\sqrt{n} = \sigma/20$. The 42 Hz hardware filter correlates neighbouring samples taken a few milliseconds apart, so the true error is somewhat larger. The estimate is only valid if the robot is still for those two seconds.

3.2.2 Zero-velocity update

In `main.cpp` the bias is refined while the robot is commanded to stand still. After one second of "still", and only while the corrected rate satisfies $|g_z| < 0.05\,\text{rad/s}$, every 50 ms sample updates

$$\hat{b}_{k+1} = \hat{b}_k + \lambda\, g_{z,k}, \qquad g_{z,k} = \text{raw}_k - \hat{b}_k, \quad \lambda = 0.02, \tag{3.3}$$

which is an exponential average of the raw readings with time constant $\Delta t/\lambda = 2.5\,\text{s}$. For white noise of standard deviation σ per sample, the estimate fluctuates around the truth with variance

$$\text{Var}(\hat{b}) = \frac{\lambda}{2-\lambda}\,\sigma^2 \approx 0.0101\,\sigma^2, \qquad \text{std}(\hat{b}) \approx 0.1\,\sigma. \tag{3.4}$$

So the residual bias after a long stop is about a tenth of the per-sample noise. Stage 1 of the test prints that noise, which makes the figure checkable: with $\sigma = 3 \times 10^{-3}\,\text{rad/s}$, $\text{std}(\hat{b}) = 3 \times 10^{-4}\,\text{rad/s}$ and about 1° per minute of later motion.

Two properties of the gate matter. First, it is keyed on the *commanded* state ($|v_{\text{cmd}}| < 0.003\,\text{m/s}$ and $|\omega_{\text{cmd}}| < 0.01\,\text{rad/s}$), not on measured stillness, so a slow push below 0.05 rad/s while the robot is commanded still is partly absorbed as bias. Second, the $|g_z| < 0.05\,\text{rad/s}$ condition means the loop can only *refine* a boot calibration that is already within 0.05 rad/s (2.9°/s); if the boot estimate is worse than that, the update never runs (Section 6.7).

3.2.3 Dead-band

After the ZUPT, `main.cpp` sets $g_z = 0$ whenever $|g_z| < 0.005\,\text{rad/s}$ (0.29°/s). At rest this removes the integrated noise and the small residual bias, which is why the displayed yaw stays frozen. It has two costs: real rotation slower than 0.005 rad/s is lost (up to 17° per minute), and, importantly for this report, the dead-band hides a residual bias only while the robot is still. During a turn $|g_z|$ is far above the threshold, so the bias is integrated in full. A yaw display that does not move at rest therefore does not prove that the bias is zero (Chapter 8).

3.3 Accelerometer tilt sensing

Inverting Equation (2.1) gives the tilt of a static robot:

$$\phi_a = \text{atan2}(a_y,\, a_z), \qquad \theta_a = \text{atan2}(-a_x,\, \sqrt{a_y^2 + a_z^2}), \tag{3.5}$$

which are the expressions in `attitude.h`. The accuracy is limited by three things:

- **Offsets.** An offset δ on a_x is a pitch error of δ/G: $0.1\,\text{m/s}^2$ is 0.58°.
- **Linear acceleration.** Any acceleration along x looks like pitch: $0.5\,\text{m/s}^2$ is 2.9°.
- **One-pose calibration.** The static offsets in Table 2.5 come from a single resting pose. They absorb any tilt of the surface the robot stood on, and for a_z they cannot separate offset from gain. A six-pose tumble would separate them; the single pose is adequate for a robot on mostly level floors.

An offset of $0.1\,\text{m/s}^2$ also integrates to 0.1 m/s of velocity error *per second*, which is why the accelerometer is never integrated on its own here: the encoders correct the velocity and a ZUPT sets it to zero at rest (Chapter 6).

3.4 Euler-angle kinematics

With body rates p, q, r and ZYX Euler angles, the angle rates are

$$\dot{\phi} = p + (q \sin\phi + r \cos\phi) \tan\theta, \tag{3.6}$$

$$\dot{\theta} = q \cos\phi - r \sin\phi, \tag{3.7}$$

$$\dot{\psi} = (q \sin\phi + r \cos\phi) / \cos\theta. \tag{3.8}$$

`attitude.h` integrates the first two with a forward Euler step and leaves the third to the EKF. The term $\tan\theta$ is singular at $\theta = \pm 90°$, which a ground robot never approaches; at the 6° ramp threshold it is 0.105.

3.5 The complementary filter

The gyro integral is accurate at short times and drifts at long times; the accelerometer angle is noisy and disturbed at short times and unbiased at long times. A first-order complementary filter weights them by frequency. With $\alpha = \tau/(\tau + \Delta t)$ one step is

$$\hat{x}_{k+1} = \alpha\,(\hat{x}_k + \omega_k\,\Delta t) + (1 - \alpha)\,x_{a,k}. \tag{3.9}$$

In continuous time this is a low-pass filter $1/(\tau s + 1)$ on the accelerometer angle plus a high-pass filter $\tau s/(\tau s + 1)$ on the integrated gyro, which add to one. The crossover is at

$$f_c = \frac{1}{2\pi\tau} = 0.159 \text{ Hz} \quad (\tau = 1 \text{ s}), \qquad \alpha = 0.952 \quad (\Delta t = 0.05 \text{ s}). \tag{3.10}$$

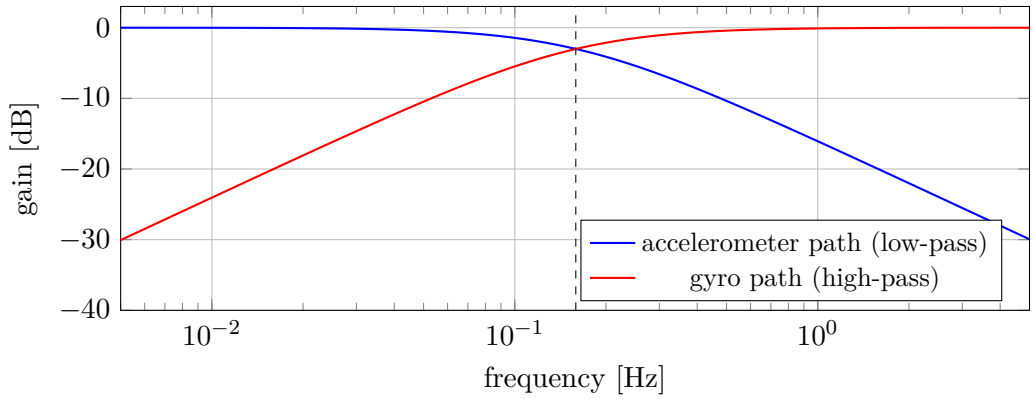


Figure 3.1: Magnitude of the two paths of the complementary filter for $\tau = 1\,\text{s}$. They cross at 0.159 Hz (dashed), where each contributes −3 dB.

3.5.1 Effect of a gyro bias

With a constant residual rate bias b on the gyro and a true angle of zero, Equation (3.9) settles at

$$\hat{x}_\infty = \frac{\alpha\, b\, \Delta t}{1 - \alpha} = \tau\, b, \tag{3.11}$$

exactly, for any Δt. A bias of 0.01 rad/s therefore costs 0.57° of roll or pitch. The Python replica of `attitude.h` (Chapter 8) reproduces this to six digits. `attitude.h` subtracts a boot-time mean for g_x and g_y and never updates it afterwards, so a drift of those two biases with temperature enters roll and pitch as $\tau\,\Delta b$.

3.5.2 Accelerometer gating

When the measured magnitude differs from G by more than 15 %, i.e. $|\|\mathbf{a}\| - G| \geq 0.147\,G = 1.47\,\mathrm{m/s^2}$, the filter sets $\alpha = 1$ and runs on the gyro alone. That catches bumps and impacts. It does not catch the accelerations this robot produces: at its top speed of 0.48 m/s the accelerations are a few tenths of $\mathrm{m/s^2}$, well inside the gate. Those leak into the pitch estimate, as the next section shows.

3.6 Gravity compensation and its side effect

The forward acceleration used by the EKF is the measured a_x with the gravity component of Equation (2.1) removed:

$$a_{\mathrm{fwd}} = a_x + G \sin \hat{\theta}. \tag{3.12}$$

This is correct if $\hat{\theta}$ is the true pitch. But $\hat{\theta}$ is itself corrected by the accelerometer angle $\theta_a = \mathrm{atan2}(-a_x, \dots)$, which includes the robot's own acceleration. Under a sustained forward acceleration a on a level floor the filter output tends to $\hat{\theta} \to -(a/G)\,(1 - e^{-t/\tau})$, and then

$$a_{\mathrm{fwd}} \approx a\, e^{-t/\tau}. \tag{3.13}$$

The compensated acceleration is the true one passed through a high-pass filter with time constant $\tau = 1\,\mathrm{s}$. Integrated, it can deliver at most $a\tau$ of velocity, however long the acceleration lasts; and when the acceleration ends, the pitch estimate still holds the leaked angle, so a_{fwd} goes negative and cancels most of what was gained. Figure 3.2 shows the result for $a = 0.3\,\mathrm{m/s^2}$ applied for four seconds.

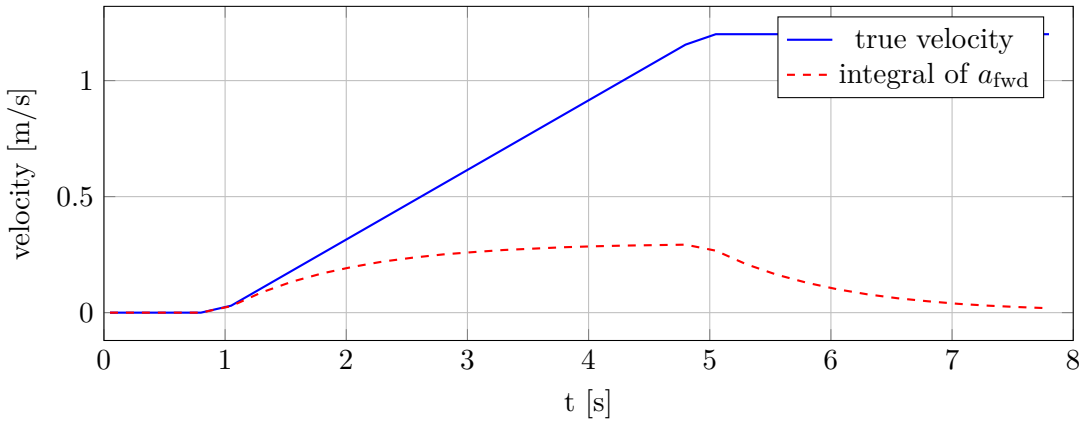


Figure 3.2: Simulation: velocity from integrating the compensated forward acceleration (dashed) against the true velocity, for $0.3\,\mathrm{m/s^2}$ applied between 1 and 5 s.

The consequence is limited by design: the EKF takes velocity mostly from the encoders (measurement noise 0.02 against process noise 0.05 for the velocity state), so the accelerometer acts only as a short-term predictor. But the accelerometer should not be relied on to supply speed during sustained acceleration, and its published `linear_acceleration` is the raw measurement scaled by k, not this compensated value.

3.7 Heading integration and the EKF

The EKF state is $[x,\ y,\ \psi,\ v]$. The prediction step advances the heading with $\psi \leftarrow \psi + g_z\,\Delta t$ and the measurement step compares predicted position, heading and velocity with encoder-derived

values. For heading, `main.cpp` forces the innovation to zero (`Y(2) = 0`), with the comment that skid-steer encoders under-read turns. One could still ask whether the encoders can influence heading indirectly, through cross-covariances in P and the velocity innovation. The Python replica of the EKF block (Chapter 8) says they do not in any measurable way: over 60 s of driving at 0.2 m/s, with a noisy encoder and with turn rates of 0, 0.3 and -1.0 rad/s, the EKF heading never differed from the pure integral of g_z by more than 1.2×10^{-6} rad. So Equation (3.2) is the complete heading error budget.

3.8 Error budget

Table 3.2 turns the models above into numbers for a single turn and for a minute of driving. The right-hand column uses the values measured in Chapter 8.

Table 3.2: Heading error budget. ψ is the angle turned and T the elapsed time.

Source	Formula	Example
Scale error	$s\,\psi$	0.43 %: 1.5° per 360°
Residual bias	$(b - \hat{b})\,T$	1.1×10^{-3} rad/s: 3.9° in 60 s
Noise (ARW)	$N_{\mathrm{ARW}}\sqrt{T}$	0.04° in 60 s
ZUPT jitter	$0.1\,\sigma\,T$	$\sigma = 3 \times 10^{-3}$ rad/s: 1° per minute of motion
Dead-band loss	up to $0.005\,T$	only for rotation slower than 0.29°/s

Scan matching in the SLAM stack absorbs errors of this size as long as the scene has structure to match, so none of these terms needs to be eliminated, but the scale and bias terms are the ones worth measuring because they are systematic.

Chapter 4

Test Firmware: `IMU_CALIBRATE.cpp`

4.1 Purpose and design

`IMU_CALIBRATE.cpp` is a guided test. The terminal tells the operator what to do and when, the firmware measures, and it prints a set of lines that all begin with `RESULT`. The operator copies those lines and nothing else needs to be recorded. It is flashed *instead of* `main.cpp`, contains no motor or ROS code, and uses the same `MPU6050_Custom` driver as the robot.

That last point shapes what the test measures. Because it calls `readData`, every value it prints is *after* the driver's static offsets of Table 2.5. It therefore measures the *residual* with respect to the stored constants, not the raw sensor output, and it does not call `calibrateGyro`, so its g_z is the raw reading with the runtime bias at its initial value of zero. Section 7.4 turns these residuals into new constants.

The test has four stages and takes about two minutes:

1. **Still** (8 s): offsets, noise, gyro bias, apparent mounting tilt.
2. **Spin** (30 s window): one full counter-clockwise turn by hand; gyro sign and scale.
3. **Nose up**: accelerometer against gyro on the pitch axis.
4. **Left side up**: accelerometer against gyro on the roll axis.

4.2 Building and flashing

The header comment of the file gives the PlatformIO environment. With the file's actual name it reads:

```
[env:imu_test]
extends = env:<your existing env>
build_src_filter = -<*> +<IMU_CALIBRATE.cpp> +<MPU6050_Custom.cpp>
```

The header comment still says `imu_guided_test.cpp`; if you copy the filter from there, the build finds no source file. The program needs `pins.h` and the driver from the project, and the serial monitor must be at 115200 baud. The prompt "press ENTER when ready" is satisfied by *any* received byte, so the monitor must be set to send a line ending (or any key) when you press Enter.

4.3 Constants and data structures

Table 4.1: Constants and state in `IMU_CALIBRATE.cpp`.

Name	Meaning
`SAMPLE_US` = 20000	Sample period, 20 ms (50 Hz)
`DUMP_RAW` = 0	If 1, every sample is also streamed as `RAW,stage,millis,ax,ay,az,gx,gy,gz`; produces long output
`struct Win`	One recording window: count n, sums and sums of squares of the six channels, and the integral of (gyro − bias) about each axis
`bias[3]`	Gyro means from Stage 1 (rad/s); subtracted in the gyro integrals of the later stages
`stageNo`	Stage number, used only for the raw dump

4.4 Sampling and statistics

`collect(ms, w)` polls in a loop with a 1 ms delay and takes a sample as soon as 20 ms have passed since the previous one, so the actual period is 20 to 21 ms. If a read fails it waits 1 ms and tries again; the interval used for integration is always the *measured* one, taken from `micros()`, so a failed read lengthens one step but does not bias the integral. For each accepted sample i with measured interval Δt_i:

$$\bar{x}_c = \frac{1}{n}\sum_i x_{c,i}, \qquad \sigma_c = \sqrt{\frac{1}{n}\sum_i x_{c,i}^2 - \bar{x}_c^2}, \qquad \Theta_k = \sum_i (g_{k,i} - \beta_k)\Delta t_i, \tag{4.1}$$

where c runs over the six channels, β_k is the Stage-1 bias of gyro axis k and Θ_k the integrated angle. The standard deviation is the population form and is computed in double precision from float inputs, which is adequate for these magnitudes. The integral is a rectangular sum with each sample weighted by the interval that ends at it.

4.5 Start-up probes

Before the stages, `setup()` identifies the chip and probes its registers. These lines are the quickest way to find out whether a board sold as an MPU6050 is one.

Table 4.2: Start-up probe lines and their interpretation. Register behaviour is from my recollection of the datasheets and register maps; check them.

Line	Meaning
`WHO_AM_I=0x68`	A genuine MPU6050. 0x70 and 0x71 are the MPU-6500 and MPU-9250 families, and 0x72 appears on some clones. `healthCheck` accepts all four, so it does not distinguish them
`WHO_AM_I read FAILED`	No acknowledge at 0x68: wiring, power, or the address pin is high (address 0x69)
`GYRO_CONFIG`, `ACCEL_CONFIG`	Should both be 0x00 (full scale ±250°/s and ±2 g, matching the 131 and 16384 divisors in the driver)
`ACCEL_CONFIG2 (1D)` readback after writing 0x03	`0x00` suggests no such register, as on a real MPU6050; `0x03` means the chip implements `ACCEL_CONFIG2`, i.e. it is in the MPU-6500 family. In that case the accelerometer filter is set by register 0x1D, which `begin()` never writes
`imu.begin() FAILED`	The wake-up write was not acknowledged; the test stops

The probe writes `PWR_MGMT_1`=0x01 before reading, then restores 0x1D to 0x00; afterwards `imu.begin()` writes `PWR_MGMT_1`=0 again, so the sensor is left in the driver's normal configuration for the stages.

4.6 Stage 1: still

Procedure. The robot rests flat on a level surface, wheels down; after a three-second countdown the firmware records for 8 s while the operator does not touch the robot or the table.

Computation. The window means of the gyro channels become the bias used by every later stage. The program prints the sample count (about 390 to 400), the mean accelerometer vector and its magnitude, the accelerometer standard deviations, the gyro means and standard deviations, and two mounting angles computed from the mean accelerometer vector:

$$\text{climb} = \operatorname{atan2}(\bar{a}_x, \sqrt{\bar{a}_y^2 + \bar{a}_z^2}), \qquad \text{leftUp} = \operatorname{atan2}(\bar{a}_y, \bar{a}_z). \tag{4.2}$$

They are positive for nose up and left side up respectively, i.e. the opposite sign to the pitch of Equation (3.5) (climb equals $-\theta_a$). On a level surface both should be near zero; a nonzero value is a mount tilt, a tilted table, or an accelerometer offset, and the test cannot tell which.

What the numbers mean. With the static offsets applied, $\bar{a}_x \approx \bar{a}_y \approx 0$ and $\bar{a}_z \approx G$, and $\|\bar{\mathbf{a}}\| \approx G$; deviations are residual offsets, or a gain error if the magnitude is wrong. The gyro means for x and y are the residuals relative to c_{gx}, c_{gy} and should be small. The z mean is the board's actual zero-rate output, which `calibrateGyro` re-measures at each boot. The three gyro standard deviations are the noise figure used in Equation (3.4). If any exceeds 0.01 rad/s the firmware prints a warning that the robot was moving or vibrating and the stage must be repeated.

4.7 Stage 2: spin

Procedure. The operator marks the robot's nose on the surface, then, after the countdown, turns the robot *one full turn counter-clockwise* seen from above, flat and smoothly, in about 6 to 10 s, ends exactly on the mark and holds still until "STOP". The recording window is 30 s regardless.

Computation. The z integral of Equation (4.1) is the angle Θ_z the gyro reports for what should be +360°:

$$\Theta_z = \sum_i (g_{z,i} - \beta_z)\Delta t_i, \qquad S_{\text{rep}} = \frac{|\Theta_z|}{360^\circ}. \tag{4.3}$$

The printed "gyro scale factor" is S_{rep}, the ratio of *reported to true* rotation. A value above 1 means the gyro over-reads, and the correction to apply is its reciprocal. The sign line says that $\Theta_z > 0$ for a counter-clockwise turn means the frame is standard (FLU), and a negative value means the axis is inverted. The program also prints the x and y integrals, which should be near zero for a flat turn, and warns if $|\Theta_z|$ is outside 270 to 450°.

Resolution. One turn cannot resolve a scale error as small as the one measured on this robot. Three effects each contribute: the operator's alignment on the mark (± 1 to 2°, i.e. ± 0.3 to $0.5\,\%$ of 360°); the residual bias integrated over the 30 s window (10^{-3} rad/s gives 1.7°, or $0.5\,\%$); and the extra rotation or lag at the start and end. Together they are of the order of $\pm 0.7\,\%$. The stage is excellent for the *sign* and for catching a gross scale error, but to measure the scale to a few tenths of a percent use the multi-turn protocol of Section 7.5.

4.8 Stages 3 and 4: tilt

Procedure. The robot lies flat; after the countdown the operator holds it flat for 3 s, lifts the front (stage 3) or the left side (stage 4) by about 15 to 20° over 4 s, and holds the raised angle for 3 s. A book under the front or the left wheels is convenient.

Computation. With the accelerometer angle A taken from Equation (4.2) (climb for stage 3, leftUp for stage 4), the program computes the change between the flat and the held windows, $\Delta A = A_{\text{hold}} - A_{\text{flat}}$, and compares it with the gyro integral over the lift and the hold windows, in the same sign convention:

$$\text{stage 3: } G_\theta = -\frac{180^\circ}{\pi}(\Theta_y^{\text{lift}} + \Theta_y^{\text{hold}}), \qquad \text{stage 4: } G_\phi = +\frac{180^\circ}{\pi}(\Theta_x^{\text{lift}} + \Theta_x^{\text{hold}}), \tag{4.4}$$

and prints the ratio $G/\Delta A$ if $|\Delta A| > 1^\circ$. The minus sign in stage 3 is there because nose up is a negative rotation about $+y$. It also prints the changes of the three accelerometer means, the gyro standard deviations while holding, and two warnings: "tilt under 8°" and "SIGN MISMATCH between accel and gyro".

Expected values. For a lift angle β the accelerometer changes by

$$\Delta a_{\text{lift}} \approx G \sin\beta, \qquad \Delta a_z \approx G(\cos\beta - 1), \tag{4.5}$$

with $\Delta a_{\text{lift}} = \Delta a_x$ in stage 3 and Δa_y in stage 4. For $\beta = 15^\circ$ these are $+2.54$ and $-0.33\,\text{m/s}^2$; for 20°, $+3.35$ and $-0.59\,\text{m/s}^2$. The gyro/accelerometer ratio should be close to 1. This stage exists to prove that the gyro and accelerometer agree in *sign and gain*, which the complementary filter of Chapter 5 depends on: a sign error in either would make the filter pull the estimate away from the truth instead of toward it.

4.9 Output reference

Table 4.3: The `RESULT` lines, in the order they appear.

Line	Content	Use
`WHO_AM_I`, `REG`	Chip identity and register values (Table 4.2)	Genuine-part check
`S1 samples`	Sample count in 8 s	Check about 390–400
`S1 accel mean`	$\bar{a}_x, \bar{a}_y, \bar{a}_z$ and $\|\bar{\mathbf{a}}\|$	Offset constants, gain check
`S1 accel std`	σ per accelerometer axis	Noise
`S1 gyro bias`	$\beta_x, \beta_y, \beta_z$	Gyro offset constants
`S1 gyro std`	σ per gyro axis	Input to Equation (3.4)
`S1 apparent climb`	Mount tilt (climb, leftUp)	Level check
`S2 gz integral`	Θ_z for one CCW turn	Scale
`S2 gyro scale factor`	S_{rep}, reported over true	Scale (take reciprocal to correct)
`S2 gx,gy integrals`	Cross-axis leakage	Mounting
`S2 verdict`	Sign of g_z for CCW	Frame check
`S3/S4 accel angle`	Flat and raised angle, change	Tilt actually applied
`S3/S4 gyro integral`	G_θ or G_ϕ	Agreement with accelerometer
`S3/S4 ratio`	$G/\Delta A$	Should be near 1
`S3/S4 raised-minus-flat`	$\Delta a_x, \Delta a_y, \Delta a_z$	Compare with Eq. (4.5)
`S3/S4 gyro std`	Noise while holding	Hand steadiness

After the last stage the program prints “ALL DONE”; pressing any key restarts the board for another run (`loop()` calls `ESP.restart()` on the first received byte).

Chapter 5

The Attitude Filter: `attitude.h`

5.1 Interface

`attitude.h` is a small header-only class that estimates roll and pitch, provides gravity-compensated forward acceleration and flags a ramp. Table 5.1 lists its members.

Table 5.1: Public interface of the `Attitude` class.

Member	Description
`roll`, `pitch`	Estimates in radians; pitch positive nose down, roll positive left side up
`ramp`	True while the robot is judged to be on a ramp
`calibrate(a[3], g[3])`	Sets the accelerometer gain, the gyro x, y bias and the zero attitude from still averages
`update(ax,ay,az,gx,gy,gz,dt)`	One filter step (Section 5.3)
`forwardAccel()`	$a_x + G\sin\theta$, the gravity-compensated forward acceleration
`gainK()`	The accelerometer gain correction k
`gyroBias(i)`	The stored bias of gyro axis i (the z entry is always 0)

5.2 Calibration

`calibrate` receives the mean accelerometer vector $\mathbf{a}$ and mean gyro vector $\mathbf{g}$ over a still period (100 samples in `attitudeCalibrate`, taken from `readData` so the static offsets are already applied).

$$k = \frac{G}{\|\mathbf{a}\|}, \qquad \hat{b}_x = \bar{g}_x, \qquad \hat{b}_y = \bar{g}_y, \qquad \hat{b}_z = 0, \tag{5.1}$$

$$\theta_0 = \operatorname{atan2}\!\left(-ka_x, \sqrt{(ka_y)^2 + (ka_z)^2}\right), \quad \phi_0 = \operatorname{atan2}(ka_y,\, ka_z), \quad \phi = \theta = 0. \tag{5.2}$$

The factor k is a single-point correction of the accelerometer gain, applied to all three axes at every update and also to the published `linear_acceleration`. The offsets θ_0, ϕ_0 define the robot's pose at boot as zero roll and zero pitch, so a robot that boots on a slope reports zero tilt there and its pitch is relative to that slope. The z bias is left to the driver and the ZUPT.

5.3 The update step

Each call performs the following, with $p = g_x - \hat{b}_x$, $q = g_y - \hat{b}_y$, $r = g_z$ (already corrected by the driver):

1. Scale the accelerometer by k and store the scaled a_x for `forwardAccel`.
2. Integrate the Euler rates of Equation (3.8): $\phi_G = \phi + (p + (q\sin\phi + r\cos\phi)\tan\theta)\Delta t$ and $\theta_G = \theta + (q\cos\phi - r\sin\phi)\Delta t$.
3. Compute the accelerometer angles, $\phi_A = \operatorname{atan2}(a_y, a_z) - \phi_0$ and $\theta_A = \operatorname{atan2}(-a_x, \sqrt{a_y^2 + a_z^2}) - \theta_0$.
4. Choose the weight: $\alpha = \tau/(\tau + \Delta t)$ if $|\|\mathbf{a}\| - G| < 0.15\,G$, otherwise $\alpha = 1$.
5. Blend: $\phi \leftarrow \alpha\phi_G + (1-\alpha)\phi_A$, and the same for θ.
6. Update the ramp flag (next section).

Listing 5.1 shows the corresponding code.

Listing 5.1: Core of `Attitude::update()`.

```
float p = gx - gb_[0], q = gy - gb_[1], r = gz;
float sr = sinf(roll), cr = cosf(roll), tp = tanf(pitch);
float rollG  = roll  + (p + (q*sr + r*cr)*tp) * dt;
float pitchG = pitch + (q*cr - r*sr) * dt;

float n = sqrtf(ax*ax + ay*ay + az*az);
float alpha = (fabsf(n - G) < 0.15f*G) ? TAU/(TAU + dt) : 1.0f;
float rollA  = atan2f(ay, az) - roll0_;
float pitchA = atan2f(-ax, hypotf(ay, az)) - pitch0_;
roll  = alpha*rollG  + (1.0f - alpha)*rollA;
pitch = alpha*pitchG + (1.0f - alpha)*pitchA;
```

Table 5.2: Constants in `attitude.h`.

Name	Value	Role
`G`	$9.80665\,\mathrm{m/s^2}$	Gravity
`TAU`	1.0 s	Complementary-filter time constant; $f_c = 0.159\,\mathrm{Hz}$
gate	$0.15\,G$	Accelerometer magnitude window for blending
`RAMP_ON`	0.1047 rad (6.0°)	$\|\theta\|$ above this starts the ramp timer
`RAMP_OFF`	0.0698 rad (4.0°)	$\|\theta\|$ below this starts the release timer
debounce	0.3 s	Time a condition must hold before the flag changes

5.4 Ramp detection

The ramp flag is a hysteresis comparator on $|\theta|$ with a debounce timer. While the flag is false, a timer accumulates Δt for as long as $|\theta| > 6°$ and is reset to zero otherwise; after 0.3 s the flag becomes true. While it is true, the timer accumulates for as long as $|\theta| < 4°$, and the flag clears after 0.3 s. The 2° hysteresis and the 0.3 s debounce keep the flag from chattering when the pitch estimate hovers near a threshold. In the supplied `main.cpp` the flag is computed but not read anywhere, so it currently has no effect on behaviour.

5.5 Behaviour checked by simulation

A Python replica of the class, written line by line from the header, was used to test three properties. They are simulations of the filter with ideal sensors, not measurements.

- **Tracking.** With a gyro and accelerometer consistent with a 10° nose-up ramp over one second, the pitch estimate follows the truth with no error beyond numerical precision, as expected when sensors agree.
- **Bias.** A residual rate bias b on g_y with a still, level robot settles at the pitch error τb predicted by Equation (3.11) (Table 5.3).
- **Acceleration leakage.** Equation (3.13), shown in Figure 3.2.

The weight α also depends on the sample period: 0.980 at the test firmware's 20 ms, 0.952 at the robot's 50 ms, and 0.909 if a step were stretched to 100 ms. The filter is not sensitive to this.

Table 5.3: Steady-state pitch error caused by a residual gyro bias (level, still robot, $\tau = 1$ s). The simulated value equals the prediction τb.

Residual bias b [rad/s]	Predicted τb [deg]	Simulated [deg]
0.001	0.057	0.057
0.002	0.115	0.115
0.005	0.286	0.286
0.010	0.573	0.573

5.6 Limitations

- Forward-Euler integration of the angles is adequate at 20 Hz for the small angles of a ground robot but would not be for fast, large rotations.
- $\tan\theta$ is singular at $\pm 90°$; harmless here.
- The x and y gyro biases are measured once at boot and never updated.
- The accelerometer is calibrated at one pose: gain and offset on each axis cannot be separated.
- The gate of 15 % of G does not exclude the robot's own accelerations (Section 3.6).
- The estimate has no yaw; the heading comes from the EKF.

Chapter 6

Integration in `main.cpp`

6.1 Boot

The relevant part of `setup()` is

Listing 6.1: IMU start-up in `setup()`.

```
if (imuSensor.begin()) {
  imu_active = true;
  imuSensor.calibrateGyro();   // robot must be still (~1.2 s)
  attitudeCalibrate();         // robot must be still and level (~2 s)
  last_filter_us = micros();
}
```

`attitudeCalibrate` reads 100 samples 20 ms apart through `readData`, averages them, and hands the means to `att.calibrate`. It returns without calibrating if fewer than 50 reads succeeded, in which case `k_` stays at 1 and the zero offsets at 0.

6.2 Processing order in the 20 Hz loop

Every 50 ms `loop()` runs the block of Table 6.1 when a sample is available and the interval is between 0 and 0.5 s.

Table 6.1: Order of operations on each IMU sample in `main.cpp`.

#	Step	Detail
1	`readData`	Driver-corrected a and g; g_z has the runtime bias removed
2	Δt from `micros()`	Used by every later step
3	`att.update(...)`	Receives g_z *before* the ZUPT and dead-band, so roll and pitch are unaffected by them
4	Still detection	Commanded velocity below 0.003 m/s and commanded turn rate below 0.01 rad/s
5	ZUPT	After 1 s of "still" and with $\lvert g_z \rvert < 0.05$ rad/s: update the bias by $0.02\, g_z$ and remove the same amount from this sample
6	Dead-band	$\lvert g_z \rvert < 0.005$ rad/s becomes 0
7	Encoder deltas	Wheel distances; used for the velocity measurement
8	EKF prediction	$\psi \leftarrow \psi + g_z \Delta t$; $v \leftarrow v + a_{\text{fwd}} \Delta t$ (forced to 0 when "still")
9	EKF update	Heading innovation forced to zero; velocity corrected by the encoders
10	Heading hold	Uses ψ and g_z when active
11	Publish	`/imu_data` and `/odometry/filtered`

6.3 ZUPT and dead-band code

Listing 6.2: ZUPT and dead-band in the IMU block.

```
bool still = fabsf(cmd_lin) < 0.003f && fabsf(cmd_ang) < 0.01f;
if (still) {
  if (still_since == 0) still_since = current_time;
  if (current_time - still_since > 1000 && fabsf(gz) < 0.05f) {
    float current_bias = imuSensor.getGyroBiasZ();
    imuSensor.updateGyroBiasZ(current_bias + (0.02f * gz));
    gz -= (0.02f * gz);
  }
} else {
  still_since = 0;
}
if (fabsf(gz) < 0.005f) gz = 0.0f;
```

The analysis of these lines is in Section 3.5 and the preceding theory chapter: a 2.5 s time constant, a residual jitter of about one tenth of the sample noise, a gate that depends on the *commanded* state, and a dead-band that hides a residual bias only while still.

6.4 The heading path in the EKF

`main.cpp` predicts ψ from g_z and forces the heading innovation to zero. To check that no indirect path exists through the covariance matrix, the EKF block was replicated in Python with the same Q, R, Jacobian, gate and update, and driven for 60 s with a noisy encoder velocity (0.2 ± 0.02 m/s) and an encoder heading that under-reads the gyro by 30 %. Table 6.2 shows the largest difference between the EKF heading and the pure integral of g_z.

Table 6.2: Largest difference between the EKF heading and $\int g_z\,dt$ over 60 s (simulation).

Turn rate g_z [rad/s]	Max difference [rad]
+0.0	0.0e+00
+0.3	3.5e-07
-1.0	1.1e-06

The differences are of the order of 10^{-6} rad: the heading is, to numerical precision, the integral of the gyro, and Equation (3.2) is its full error budget. The row of the gain matrix for heading has entries of order 10^{-5} on the velocity innovation, which is why nothing leaks through.

6.5 Published messages

- `/imu_data`: orientation from roll and pitch of `att` and yaw from the EKF; `angular_velocity` is $g_x - \hat{b}_x$, $g_y - \hat{b}_y$ and the post-ZUPT, post-dead-band g_z; `linear_acceleration` is the *raw* measurement scaled by k (not the gravity-compensated value). Frame `imu_link`.
- `/odometry/filtered`: EKF position and heading; linear velocity v from the EKF; angular velocity g_z; frames `odom` and `base_link`.
- Covariances are constants: orientation 0.01, 0.01, 0.1 for roll, pitch, yaw; angular velocity 0.001 per axis; linear acceleration 0.05. The angular velocity value corresponds to a standard deviation of 0.03 rad/s, much larger than the sensor's noise; the pose and twist covariances in the odometry message come from the EKF matrix P, with a placeholder of 0.05 for the angular-velocity entry.

Both messages are stamped with `rmw_uros_epoch_nanos()`, which depends on the micro-ROS clock synchronisation; the clock is synchronised at connect and every five minutes.

6.6 The self-check

The self-check calls `healthCheck()` and reports pass or fail for the IMU in bits 0 and 1 of `/self_check`: the identity must be one of the four accepted values, the gravity magnitude must be 0.8 to 1.2 g and the stored z bias must be within 0.35 rad/s. It does not test the offsets, the scale, or the signs; that is the job of `IMU_CALIBRATE.cpp`.

6.7 Risk at power-up

The boot calibration has no second chance for large errors. If the robot is moved or rotated during `calibrateGyro`, the stored bias is wrong by the mean rate during those two seconds. The ZUPT can correct that only if the *corrected* rate satisfies $|g_z| < 0.05$ rad/s, i.e. if the boot error is already below 2.9°/s. A larger error blocks the update permanently, and the robot then drifts at that rate while still until it is rebooted; the dead-band hides nothing unless the drift is below 0.29°/s. Power the robot up on a stable surface and leave it alone for about four seconds. If a yaw drift at rest is ever seen after boot, reboot before investigating anything else.

Chapter 7

Calibration Procedure

7.1 Before you start

- Fully charged battery (the IMU does not depend on it, but a brown-out resets the board mid-test).
- A flat, level, rigid surface; a stable table is better than a floor that flexes.
- A marked line on the surface and a mark on the robot's nose for the turn tests.
- A book or block 20 to 30 mm thick for stages 3 and 4.
- Serial monitor at 115200 baud, set to send a line ending.
- The current stored constants (Table 2.5) written down, so that every change can be undone.

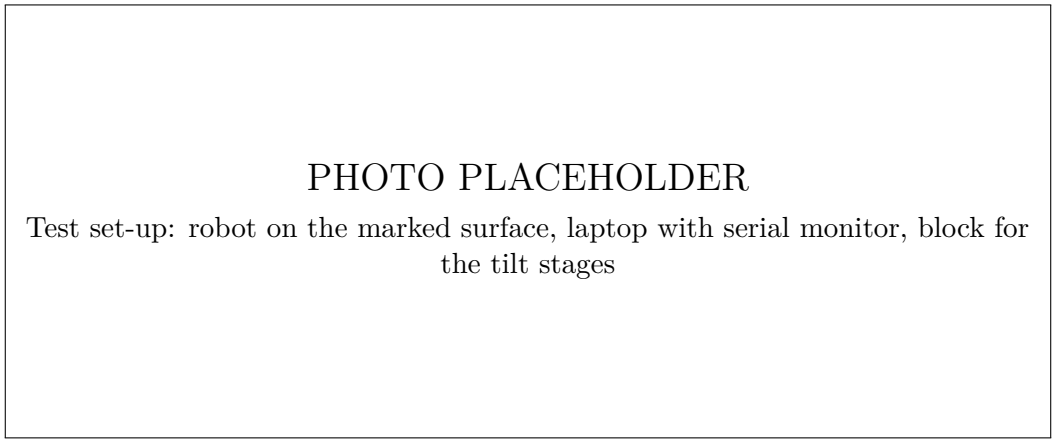


Figure 7.1: Test set-up: robot on the marked surface, laptop with serial monitor, block for the tilt stages

7.2 Running the guided test

1. Flash `IMU_CALIBRATE.cpp` (Section on building, Chapter 4). Open the monitor and note the probe lines.
2. Run all four stages in one session, following the prompts. Copy every `RESULT` line to a text file with the date.
3. Repeat the whole test twice more, rebooting between runs. Three runs show whether a number is a property of the sensor or of the operator.

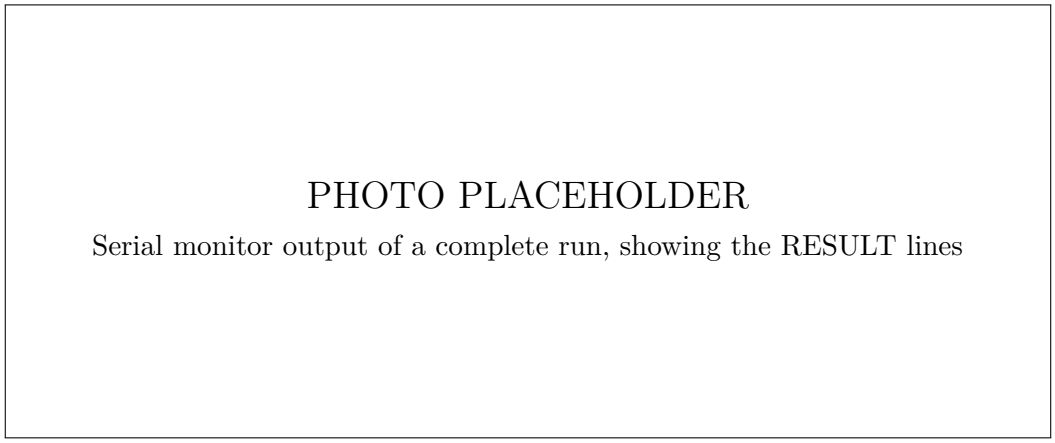


Figure 7.2: Serial monitor output of a complete run, showing the RESULT lines

7.3 Acceptance criteria

Table 7.1: Acceptance criteria for the guided test. Thresholds are practical limits chosen for this robot, not datasheet values.

Check	Pass	If it fails
`WHO_AM_I`	0x68	Part is not an MPU6050; verify scale and filter registers
Stage 1 sample count	390–400	I^2C errors or slow reads
$\|\bar{\mathbf{a}}\|$	$G \pm 0.15\,\mathrm{m/s^2}$	Gain error or wrong full-scale
$\bar{a}_x, \bar{a}_y$	$\|\cdot\| < 0.1\,\mathrm{m/s^2}$	Update offsets (Section 7.4) or level the table
Gyro x, y means	$\|\cdot\| < 0.005\,\mathrm{rad/s}$	Update offsets
Gyro std (all axes)	$< 0.01\,\mathrm{rad/s}$	Vibration; repeat
Stage 2 verdict	Standard frame (CCW gives +)	Axis inverted; the whole heading chain is wrong
Stage 2 scale S_{rep}	0.99–1.01	Apply a scale factor if repeatable
Stage 3/4 ratio	0.9–1.1	Tilt applied unevenly; repeat with a firmer lift
Stage 3/4 sign	No mismatch warning	Gyro or accelerometer axis inverted

7.4 Updating the stored constants

The test reads through `readData`, so each printed mean is a *residual* with respect to the constants already stored in the driver. A residual m on a channel with stored constant c (where the driver computes "raw $-\,c$") is removed by

$$c_{\mathrm{new}} = c + m \quad \text{(gyro } x, y;\ \text{accelerometer } x, y), \qquad c^{(a_z)}_{\mathrm{new}} = c^{(a_z)} + (\bar{a}_z - G). \tag{7.1}$$

For the accelerometer z channel the stored constant is $10.444 - G$, i.e. the driver subtracts the amount by which the raw reading exceeds G, so the *target* of the mean is G rather than zero. Only change a constant if the residual is both larger than the acceptance limit and reproducible over three runs. The gyro z offset is not stored: `calibrateGyro` measures it at each boot, so the Stage-1 z mean is informational.

Scale. If Stage 2 (or the multi-turn test below) reports a repeatable ratio S_{rep} of reported to true rotation, the corrected sensitivity is

$$\text{LSB}/(^\circ/\text{s}) \;=\; 131 \cdot S_{\text{rep}}, \tag{7.2}$$

because dividing the same counts by a larger number produces smaller angles. For the measured $S_{\text{rep}} = 1.0043$ this is 131.57. The same constant must be used in `calibrateGyro` and `readData`, otherwise the boot bias is in different units from the signal it is subtracted from. Do not also multiply g_z in `main.cpp`.

Warning. Change one constant at a time and re-run the test. If two changes are made together, a regression cannot be attributed.

7.5 Multi-turn scale test

One turn has a resolution of about $\pm 0.7\,\%$ (Section 4.7); N turns divide the same fixed errors by N. Run the robot in firmware that publishes `/odometry/filtered`, with the dashboard showing yaw.

1. Leave the robot still for at least 10 s after boot so the ZUPT has converged.
2. Align the nose with the floor line; note the dashboard yaw ψ_0.
3. Turn five full turns in one direction, smoothly and slowly, by hand or by a slow `/cmd_vel`; stop exactly on the line; note ψ_1.
4. Repeat in the other direction, starting from where you are.

The over-read is $(|\psi_1 - \psi_0| - 1800^\circ)/1800^\circ$ for each direction. Compare both directions: a scale error has the *same sign* for both (the gyro over-reads in both directions), a constant bias has the same sign in the dashboard (it adds to the angle regardless of direction) and therefore appears with *opposite* signs in the over-read of the two directions. Misalignment is random. Repeat each direction twice. To separate scale from bias, repeat at a different turn rate: bias grows with the time taken, scale with the angle.

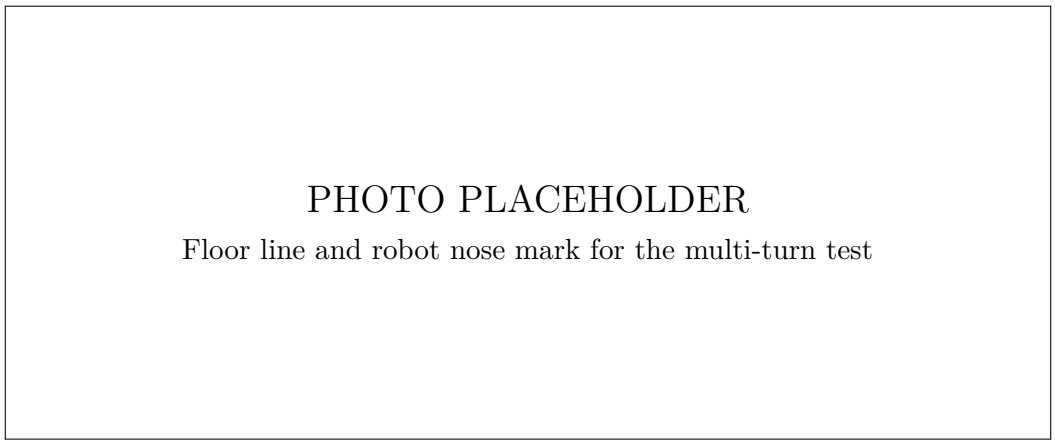


Figure 7.3: Floor line and robot nose mark for the multi-turn test

7.6 Verifying after a change

After editing constants, re-run all four stages. The residuals should be inside Table 7.1, then repeat the multi-turn test. Record before and after values in a table (Table 7.2).

Table 7.2: Commissioning record. Fill in at each calibration.

Quantity	Before	After	Date
Accelerometer offsets c_{ax}, c_{ay}			
Accelerometer z offset			
Gyro offsets c_{gx}, c_{gy}			
Gyro sensitivity [LSB/(deg/s)]			
Stage 2 scale S_{rep}			
5-turn CW over-read			
5-turn CCW over-read			
60 s still yaw drift			

Chapter 8

Validation

8.1 Hand-turn tests on the robot

The robot was turned by hand through five full turns in each direction on the floor, starting from rest after boot. The true angle was 1800°, set by a floor line; the dashboard yaw (from `/odometry/filtered`) was recorded.

Table 8.1: Five-turn tests. Single runs, floor line aligned by eye (±1 to 2°).

Direction	Dashboard change	Over-read	Relative
Clockwise	−1803.7°	3.7°	0.21 %
Counter-clockwise (start 0.3°)	+1811.9°	11.6°	0.64 %

Both directions over-read, so the gyro sensitivity is a little low relative to the nominal 131 LSB/(deg/s): the mean over-read of 7.65° is 0.43 %, and $S_{\text{rep}} \approx 1.0043$. The difference of the two runs, about 3.9° on each side of the mean in the same sign for both directions, is not explained by scale alone. Two explanations fit: a residual gyro bias, which Section 7.5 would show as an opposite-sign over-read between directions, or alignment error. A bias of 1.1×10^{-3} rad/s over a run of about 60 s would give 3.8°; this is an inference from two runs, not a measurement. Table 3.2 uses that value as its example.

A 60 s still test (dashboard yaw with the robot untouched) showed no visible change in yaw, within the 0.1° display resolution. The dead-band of 0.005 rad/s hides any residual bias below 0.29°/s *while still*, so this test cannot distinguish a small bias from none; the same bias does appear during turns, because the dead-band no longer applies to a signal that exceeds it. That observation is what the turn-rate/duration test of Section 7.5 is designed to settle.

8.2 Phone compass comparison

A phone compass held next to the robot agreed with the dashboard to within ±0.3° for counter-clockwise turns. Clockwise readings differed by up to 3° and sometimes 1.1°. A phone compass is not a reference at that level: its resolution, its response lag and the tilt of the phone each contribute errors of the same size. The floor line is the better reference.

8.3 Replica of the attitude filter

The Python replica of `attitude.h` (listed in Appendix B) confirmed the behaviour in Table 8.2. All items are simulations with ideal sensors.

Table 8.2: Simulation checks of the attitude filter and EKF heading path.

Check	Result
Tracking a 10° nose-up ramp	Maximum error 0.0° (sensors consistent)
Bias response	Equals τb (Table 5.3)
Sustained acceleration of $0.3\,\mathrm{m/s^2}$ for 4 s	True velocity gain $1.20\,\mathrm{m/s}$; integral of a_{fwd} only $0.015\,\mathrm{m/s}$
EKF heading vs gyro integral	Differences of order 10^{-6} rad (Table 6.2)

8.4 Data to collect on the robot

The following measurements are left blank for the commissioning session.

SCREENSHOT / PLOT PLACEHOLDER

Stage 2 trace: integrated gyro angle against time for one counter-clockwise turn

Figure 8.1: Stage 2 trace: integrated gyro angle against time for one counter-clockwise turn

SCREENSHOT / PLOT PLACEHOLDER

Pitch estimate against time while lifting the nose by 15° (Stage 3), with the accelerometer angle for comparison

Figure 8.2: Pitch estimate against time while lifting the nose by 15° (Stage 3), with the accelerometer angle for comparison

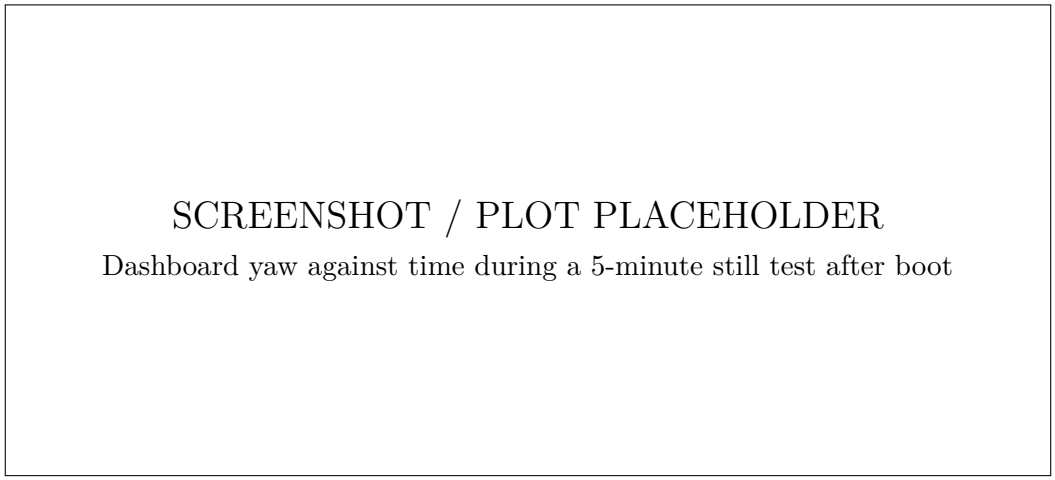


Figure 8.3: Dashboard yaw against time during a 5-minute still test after boot

Chapter 9

Troubleshooting

Table 9.1: Symptoms, likely causes and checks.

Symptom	Likely cause	Check
`imu.begin() FAILED`	Wiring, power, address pin	I^2C scan; SDA/SCL pull-ups; AD0 level
`WHO_AM_I` not 0x68	Different chip or clone	Probe lines; ACCEL_CONFIG2 readback
Stage 1 sample count low	Slow or failing reads	I^2C clock; cable length
Gyro std above 0.01 rad/s	Vibration, table moving	Repeat on a rigid surface, motors off
Gravity magnitude far from G	Wrong full-scale or gain	Read `ACCEL_CONFIG`; compare with 16384 divisor
CCW turn gives negative Θ_z	Frame inverted	Chip orientation; any sign flip in firmware
Stage 2 total much less than 360°	Turn too fast, rate beyond range, or window too short	Turn slower; ±250°/s limit
Stage 3/4 sign mismatch	Accelerometer or gyro axis inverted	Mounting orientation
Yaw drifts at rest after boot	Robot moved during calibration	Reboot and leave still for 4 s (Section 6.7)
Yaw drifts only while turning	Residual bias or scale error	Multi-turn test at two rates
Pitch/roll wrong after boot	Boot was on a slope or moved during attitude calibration	Reboot on a level surface
Robot reports motion while still	Accelerometer leak into velocity	Encoders dominate; check encoder wiring
Self-check bit 0 or 1 failing	Identity, gravity magnitude or z bias out of range	`healthCheck` limits (Section 6)

Chapter 10

Limitations and Observations

10.1 Properties of the method

- The test measures residuals relative to the stored constants; it cannot detect a mistake in the constants of the driver's *units* (for example a wrong divisor) except through the magnitude and scale checks.
- Accelerometer calibration is a single pose. Gains and offsets are not separately observable. A six-position test would separate them.
- Stage 2's resolution is about $\pm 0.7\,\%$.
- Temperature is not controlled. MEMS gyro bias changes with temperature; the boot bias is measured once, and the ZUPT follows only while the robot is still.
- No magnetometer: absolute heading cannot be corrected.

10.2 Findings in the code

Each item is something observed while reading the files supplied; none was tested on hardware.

1. The header comment of `IMU_CALIBRATE.cpp` names the file `imu_guided_test.cpp`.
2. The verdict text of Stage 2 refers to a `gz = -gz` line that does not exist in the supplied `main.cpp`.
3. The probe's "ACCEL_CONFIG2" interpretation is useful but registers 0x1D differ between families; check the datasheet.
4. `healthCheck` accepts four identities, so it will not flag an MPU-6500-family part sold as an MPU6050.
5. `att.ramp` is computed but unused in `main.cpp`.
6. The x and y gyro biases in `attitude.h` are fixed at boot.
7. The published `linear_acceleration` is not gravity-compensated; the compensated value is used internally only.
8. The gyro angular-velocity covariance in `/imu_data` (0.001) is larger than the sensor noise implies. This is conservative and acceptable.
9. A boot-time calibration error above $2.9^\circ/\mathrm{s}$ cannot be corrected by the ZUPT (Section 6.7).
10. The dead-band hides a residual gyro bias at rest but not while turning (Chapter 8).

10.3 Possible improvements

These are suggestions, not recommendations to change a working system.

- Apply the scale of Equation (7.2) once it is repeatable.
- Re-measure the gyro x, y biases when the ZUPT is active.
- Use the ramp flag to ignore accelerometer-derived velocity on slopes.
- Reject the accelerometer in the complementary filter while the commanded acceleration is nonzero.

Appendix A

Quick Reference

Table A.1: Quick reference.

Item	Value
Gyro sensitivity (nominal)	131 LSB/(deg/s), $\pm 250^\circ$/s
Accelerometer sensitivity	16384 LSB/g, $\pm 2\,g$
Boot gyro calibration	400 samples at 3 ms, accepted if at least 100 reads
Attitude calibration	100 samples at 20 ms
Filter time constant	1.0 s (0.159 Hz)
ZUPT gain / dead-band	0.02 per sample / 0.005 rad/s
EKF Q	diag(0.001, 0.001, 0.005, 0.05)
EKF R	diag(0.05, 0.05, 0.1, 0.02)
Test sample period	20 ms
Control loop period	50 ms

Appendix B

Simulation Scripts

The simulations quoted in this report were produced by `sim_imu.py`, a line-by-line Python replica of the `Attitude` class and of the EKF block of `main.cpp`. Its scenarios are: a level robot with a residual gyro y bias; a 10° nose-up ramp with consistent sensors; a sustained $0.3\,\mathrm{m/s^2}$ acceleration on a level floor; and the EKF with a noisy encoder velocity and an encoder heading that under-reads by 30 %. It uses `numpy` only. The replicas are written from the source, not from the compiled firmware, so a difference between the two would not be detected by them.